```
Output:
Found 0 matching node(s)
Found 1 matching node(s)
string(7) "project"
string(19) "software javascript"
```

The principle lesson here is that node position is always counted from `1`, not `0`, so be aware of that if you were to generate XPath expressions from looping structures. Otherwise, the XPaths above should be pretty straightforward: we navigate to `child::data` from the (implicit/virtual) document node, then `child::projects`. In that context, we then test the `position()` value of each `project` node; none are at position `0`, and one is present at position `1`. We confirm it is the first `project` element in our XML by writing out its `class` attribute. We could (and should) shorten this expression to `/data/projects/project[1]` – the `position()` function is best deployed when excluding nodes; for example, `[position() > 1]` would return all but the first node in a list.

Now for something more common: finding nodes based on class. Let's find all the `project` nodes that have a `software` class. Remember we'll need to call `contains()` rather than using an equality test because we're mimicking HTML class behavior (assigning multiple classes):

```
// ... $xml, $dom, and $xpath init as before
$results = $xpath->query('//project[contains(@class, "software")]');
print "Found {$results->length} matching node(s)" . PHP_EOL;
```

```
Output:
Found 3 matching node(s)
```

Great, we found the three `project` elements in our document that have `"software"` somewhere in their `class` attribute. Now let's pretend that we did that because we're actually more interested in the `codebase` child elements. Sure, we could use the `project` elements in the node list that we got back from our first query and iterate over their children, but there's a much neater way to do it by tweaking the XPath query:

```
// ... $xml, $dom, and $xpath init as before
$results = $xpath->query('//project[contains(@class, "software")]/
codebase');
```